Docker

LP devops

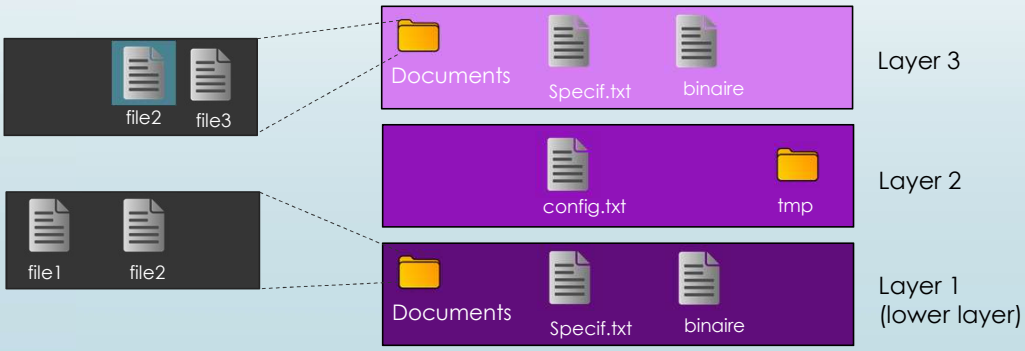
Plan

- Séance 1:
  - Cours: environnement docker / principe de base
  - TD/TP
- **Séance 2:**
  - **Cours: image et compose**
  - **TD/TP**
- Séance:
  - Cours: orchestration
  - Projet

Docker: les images

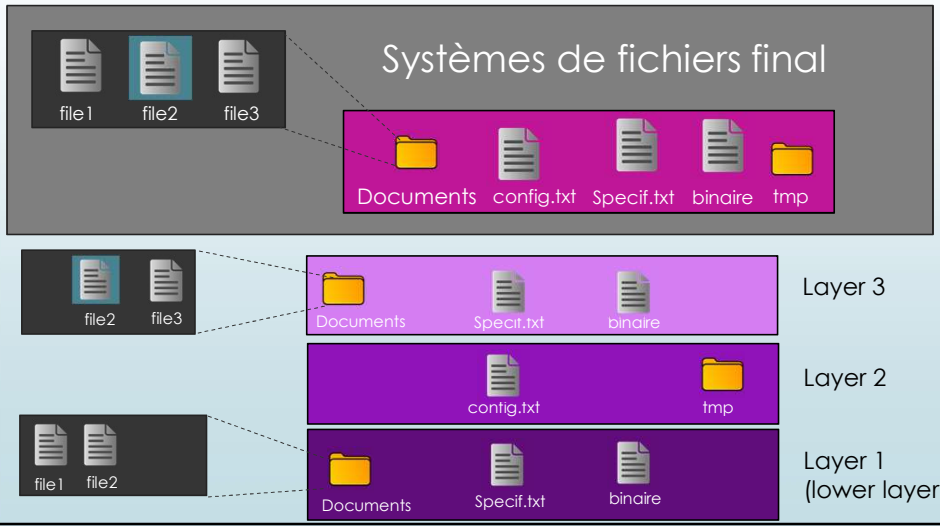
Structure des images: le système de fichiers

- Utilisation du système de fichiers overlay2
  - Anciennement overlay et aufs
  - Ils appartiennent à la famille des Union file system
- Principe:
  - Fonctionnement en couche
  - Possibilité de fusionner des dossiers



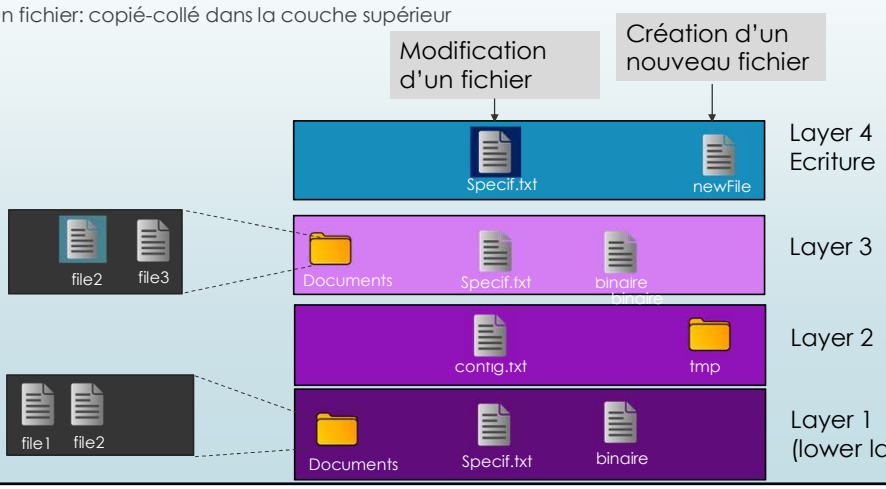
Le système de fichiers overlay2

- **Les couches sont fusionnées:**
  - **En cas de conflits les fichiers de niveaux supérieurs écrasent les fichiers de niveaux inférieurs.**

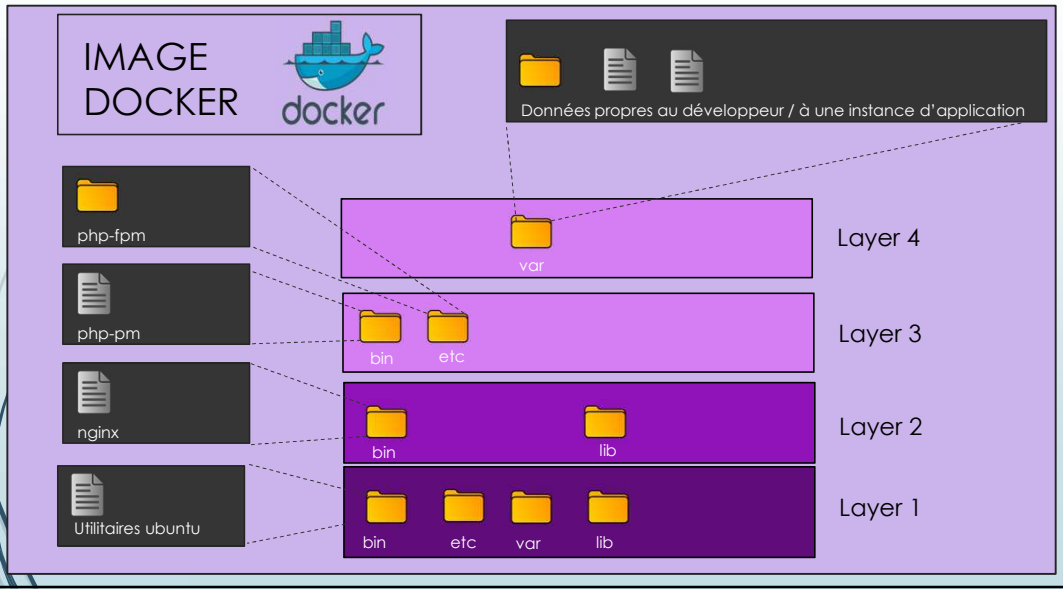


Overlay2: écriture

- Les couches inférieures sont en lecture uniquement
- Les écritures se font dans la couche de plus haut niveau
- Création de fichiers
- Modification d'un fichier: copié-collé dans la couche supérieur

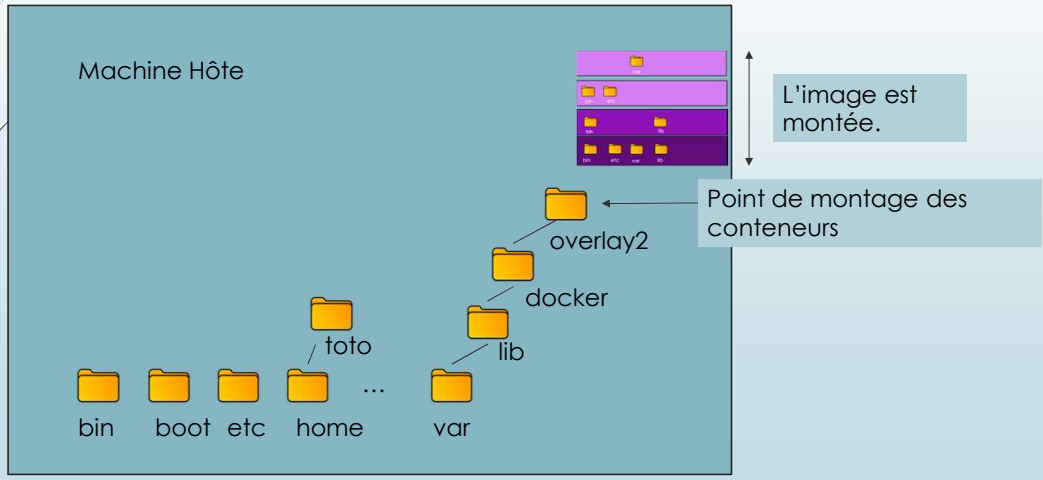


Overlay2 dans docker



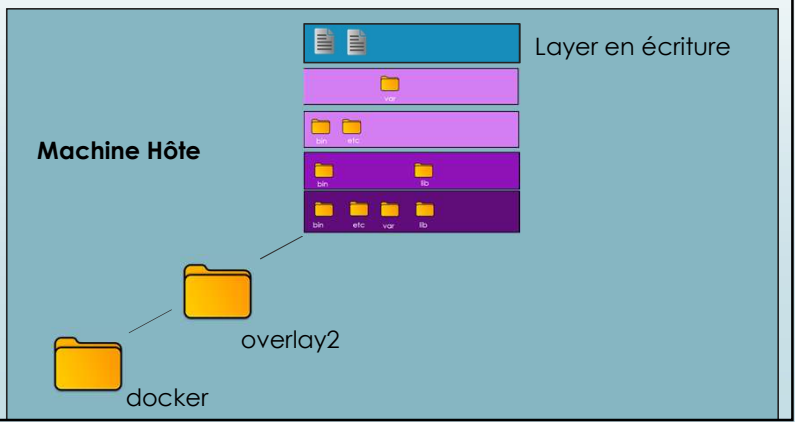
Conteneur en cours d'exécution

- Les images sont montés dans le dossier /var/lib/docker/overlay2
- Elles sont montées avec un système de fichiers overlay2



Conteneur en cours d'exécution

- Une couche en écriture est créer lors du montage qui va héberger les changement par rapport à l'image.
- L'image n'est pas sauvegardée/modifiée à la fin de la vie du conteneur.
- Les modifications ne sont pas pris en compte lors du `umount`.



Sauvegardez des modifications

- Pour sauvegarder les modifications il faut monter:
  - Un volume
  - Une branche du système de fichiers hôte
- Commande:

```
docker run -v cheminHost:cheminDansLeConteneur  image_du_conteneur
```

Docker: construction d'une image

- Ecriture d'un Dockerfile
- Les différentes couches du conteneur sont indiquées
  - Base: image existante d'un conteneur de base (ubuntu par exemple)
  - Rajout de packages/logiciels
    - Chaque logiciel/package rajoutera une couche overlay2
  - Copier coller de fichiers supplémentaires
    - Contenu d'une base de données
    - Fichiers de configuration propres (différents de ceux par défaut)
    - Etc.
- Puis construction de l'image

docker build –t <nom_image> . –f <nom_du_dockerfile>

Le dockerfile: FROM

Dockerfile

FROM ubuntu

Image de base
Le conteneur va être chrooté
Nécessité d'installer l'ensemble des binaires, configuration, et librairies nécessaire à l'exécution des programmes.

Layer 1
Ubuntu

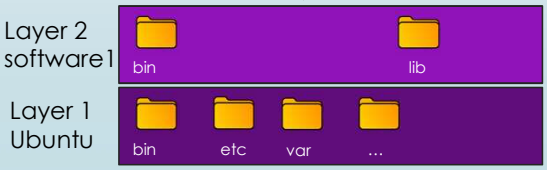
Le dockerfile: RUN

Dockerfile

```
FROM ubuntu

RUN apt-get install software1
```

Commande qui va être réalisée lors de la création de l'image. Installe un ensemble de dossiers et de fichiers supplémentaires dans une nouvelle couche.



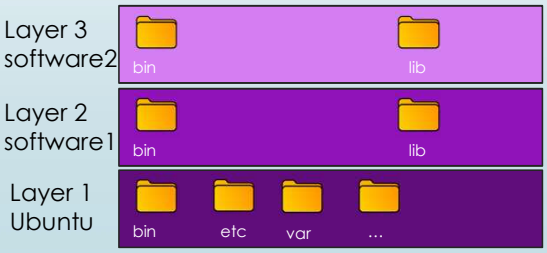
Le dockerfile: RUN

Dockerfile

```
FROM ubuntu

RUN apt-get install software1
RUN apt-get install software2
```

Chaque instruction RUN ajoute une couche au système de fichiers overlay.



Le dockerfile: RUN

Dockerfile

```
FROM ubuntu

RUN apt-get install software1
&& apt-get install software2
```

Compromis à trouver sur le nombre de couches.

Le dockerfile: RUN

Dockerfile

```
FROM ubuntu

RUN apt-get install software1
&& apt-get install software2
```

Compromis à trouver sur le nombre de couches.

Le dockerfile: COPY et ADD

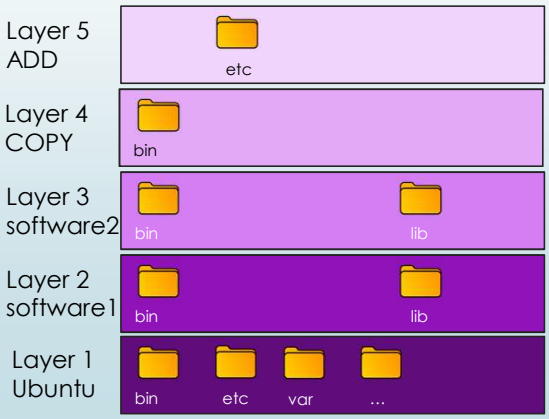
Dockerfile

```
FROM ubuntu

RUN apt-get install software1
RUN apt-get install software2
COPY  /bin/myCommand  /bin
ADD   http://127.0.0.1/index.html /etc
```

Chaque instruction COPY et ADD ajoute une couche au système de fichiers overlay.

COPY et ADD copie/colle des fichiers ou des dossiers. ADD permet en plus de spécifier des url ou des fichiers .tar comme source.



Le dockerfile: WORKDIR / EXPOSE

Dockerfile

```
FROM ubuntu

RUN apt-get install software1
RUN apt-get install software2
COPY  /bin/myCommand  /bin
ADD   http://127.0.0.1/index.html /etc

WORKDIR dossier
COPY file1  folder/
```

Change de répertoire. Utile pour l'instruction qui suit.
Ici, on entre dans dossier puis on copie le fichier file1.

Indique le port à l'écoute.

Partage un dossier avec le système de fichiers hôte.

Le dockerfile: CMD

Dockerfile

```
FROM ubuntu

RUN apt-get install software1
RUN apt-get install software2
COPY  /bin/myCommand  /bin
ADD  http://127.0.0.1/index.html /etc

WORKDIR dossier
COPY file1  /bin


CMD  /bin/software1 arg
```

Les deux seules champs/instructions obligatoires sont:
FROM

CMD ou **ENTRYPOINT**

Commande lancé au démarrage du conteneur.
Ou sous forme de liste:
CMD [« /bin/software1 », « arg1 »]

CMD et ENTRY POINT

```
FROM bash

…

#Utilisation de ENTRYPOINT et CMD


CMD  ls
```

```
bash#docker run myls
-rw-rw-r-- 1 busson busson 53 déc.   2 08:55 fichier1.txt
-rw-rw-r-- 1 busson busson 53 déc.   2 08:55 fichier2.txt
…
```

Sans argument docker lance CMD:

ls

CMD et ENTRY POINT

FROM bash

…

#Utilisation de ENTRYPOINT et CMD

CMD ls

```
bash#docker run myls echo toto
toto
```

Les arguments de docker run remplace CMD:

echo toto

CMD et ENTRY POINT

FROM bash

…

#Utilisation de ENTRYPOINT et CMD

ENTRYPOINT [« ls », »-l »]
CMD [« fichier1.txt »]

Conseil: utilisé des listes.

```
bash#docker run myls
-rw-rw-r-- 1 busson busson 53 déc.  2 08:55 fichier1.txt
```

Sans argument docker lance ENTRYPOINT+CMD:

ls –l fichier1.txt

CMD et ENTRY POINT

FROM bash

…

#Utilisation de ENTRYPOINT et CMD

ENTRYPOINT [« ls », »-l »]
CMD [« fichier1.txt »]

```
bash#docker run myls fichier2.txt
-rw-rw-r-- 1 busson busson 53 déc.  2 08:55 fichier2.txt
```

Les arguments remplacent CMD et docker lance ENTRYPOINT+CMD:

ls –l fichier2.txt

CMD et ENTRY POINT

FROM bash

…

#Utilisation de ENTRYPOINT et CMD

ENTRYPOINT [« ls », »-l »]
CMD [« fichier1.txt »]

```
bash#docker run --entrypoint=« date » myls
mer. 02 déc. 2020 09:18:34 PST
```

Docker lance ENTRYPOINT, CMD est vide:

date

CMD et ENTRY POINT

FROM bash

…

#Utilisation de ENTRYPOINT et CMD

ENTRYPOINT [« ls », »-l »]
CMD [« fichier1.txt »]

bash#docker run --entrypoint=« echo » myls toto titi
toto titi

Docker remplace ENTRYPOINT par « echo » puis CMD par « toto titi » et lance ENTRYPOINT+CMD:

echo toto titi

Questions

- On considère le conteneur Dockerfile suivant.
- Donnez le résultat des commandes:
  - docker run mycontainer
  - docker run mycontainer tata
  - docker run mycontainer titi toto
  - docker run --entrypoint=« ls » mycontainer
  - docker run --entrypoint=« ls » mycontainer tata

FROM bash

…

#Utilisation de ENTRYPOINT et CMD

ENTRYPOINT [« echo », »toto »]
CMD [« titi»]

Variables d'environnement

- Des variables d'environnement peuvent être fixées dans le Dockerfile

```
FROM bash

…

#exemples de variables d'environnement



ENV MAVAR=mavaleur
ENV LIBPATH=/lib/libMyAppli
```

Volumes

- Des volumes peuvent être spécifiés dans le Dockerfile

```
FROM bash

…

#exemple de variables d'environnement



VOLUME /var/exemple
```

Volume anonyme uniquement.
Les volumes nommés sont spécifiés au lancement du conteneur ou dans docker-compose.

Conteneur light

- Possibilité de créer une image partant de rien
- Permet d'avoir une image/conteneur léger
- Nécessité de copier-coller les binaires et librairies nécessaires dans /lib, /bin, etc.

```
FROM scratch

ADD ….
COPY ….

CMD  mybin
```

Docker ignore / docker commit

- **Docker ignore (fichier pris en compte lors de la construction de l'image)**
  - Fichier .dockerignore se trouvant dans le répertoire de travail
  - Permet d'ignorer des fichiers au moment de la construction de l'image
- **Docker commit** (commande docker)
  - Pas d'interaction possible lors de l'installation de logiciels (avec RUN par exemple)
  - Installé en mode non interactif (par exemple apt-get install –y …)
  - Utilisé la commande docker commit
    - Modifié le conteneur en mode interactif (lancez un bash dans le conteneur par exemple pour l'installation)
    - Docker commit pour mettre à jour l'image à partir de la version courante du conteneur
  - Sinon, certaines applications utilisent des variables d'environnement pour fixer le contexte ce qui permet d'éviter de demande à l'utilisateur lors de l'installation (la TIME ZONE pour apache2 par exemple).

Background / foreground

- Docker requiert que le processus principal soit en **foreground**
  - Autrement ce processus (et le conteneur) s'arrêtent
- Exemple:
  - Pour le bash: nécessité de le mettre en mode interactif (-it)
  - Pour les services en foreground dans l'image:
    - Exemple: apachectl -D FOREGROUND
- Sinon en général:
  - Pensez à lancer une commande qui reste en cours d'exécution (cela fixe la durée de vie du conteneur)
- Possibilité de lancer votre conteneur en background (detached) avec un processus en foreground à l'intérieur:
  - docker run -t -d imageName

Divers : to do

- FLOT → dépôt chez DOCKER CHART FLOW
- Docker attach
- Docker save (permet de saugarder des images docker et de les transférer sur d'autres PC sans avoir à passer par le site docker)

Les autres conteneurs / divers

- Docker-Swarm (pas un conteneur mais un orchestrateur)
- Kubernets (pas un conteneur mais un orchestrateur)
- Openshift

To Do

- Un exercice sur network dans docker-compose (en l'état les ports de tout les conteneurs sont accessibles de l'exterieur). Faire une topologie ou seul un des conteneurs est accessible de l'exterieur.
- Volume sur le réseau. En parler mais mis en place difficile en TP…

Docker compose

Docker compose

- Permet de combiner des conteneurs
  - Décrit plusieurs conteneurs interagissant les uns avec les autres
- Définit pour les différents conteneurs:
  - Les volumes
  - Les aspects réseaux
  - Les images et dockerfile
  - Les ports
  - Etc.
- S'appuie sur un fichier yaml

Yaml (« Yamelle »)

- YAML: Yet Another Markup Language
- Fichier texte/ASCII structurant un ensemble de données
- Extension .yaml ou .yml

YAML est un **langage de sérialisation de données utilisé pour écrire des fichiers de configuration dans le cadre d'un déploiement d'infrastructures**.

Exemple

Association clé valeur
clé: valeur
Attention à l'espace après le :

Espaces pour indenter et pas de tabulation.

--- marque le début du document (optionnel)

```
---
# Ceci est un commentaire
nom: toto
Champ1:
  souschamp1: valeur1
  souschamp2: 3
  souschamps3: 3.1415
  souschamps4: true
champ1:
  - azerty
  - qwerty
  - jdgeus
```

Commentaire
Mettre un espace après #

Champ1 est vu comme un dictionnaire

champ1 est un tableau

Yaml est sensible à la casse.

Les valeurs peuvent être des booléens, des entiers, des flottants, des chaînes de caractères (avec des guillemets ou non).

Docker-compose: le fichier

- Son nom par défaut est docker-compose.yml

```
version: « 3.8 »
services:
  service1:
      xxxx
      yyyy
volumes:
      zzzzz
networks:
      vvvv
```

Optionnelle mais des directives sont plus ou moins supportées suivant les versions.

Liste des services (conteneurs)

Docker-compose: services

- Les propriétés d'un service sont les suivantes:

```
version: « 3.8 »
services:
  service1:
      images: « image »
```

On utilise une image déjà existante.

```
version: « 3.8 »
services:
  service1:
      build: ./rep
```

On construit l'image à partir d'un Dockerfile dans le répertoire « rep ».

```
version: « 3.8 »
services:
  service1:
      build:
         context: rep
         dockerfile: Dockerfile-monAppli
```

On indique le répertoire où se trouve le dockerfile et un nom pour celui-ci.

Il faut soit image soit build.

Docker-compose: network

- Il y a un « network » par défaut que tous les conteneurs joignent (pas besoin de le spécifier).
  - Tous les conteneurs sont joignables entre eux.
- Les conteneurs se connectent aux autres via leurs adresses IP.
  - Un DNS résout les adresses
  - Les noms des services correspondent aux conteneurs dans le DNS
- Possibilité de créer des topologies plus spécifiques/limitées

```
version: « 3.8 »
services:
  service1:
      images: « image »
      port:
        - « 80:90 »
      networks:
        - network1
networks:
  network1:
      name: network1
```

Indique que ce conteneur joint network1

Question

- Donnez les directives network pour que 3 conteneurs soient en pipeline du point de vue du réseau.

Volumes

- Déjà vue: même principe

Attention volumes et networks définit au même niveau que services!

```
version: « 3.8 »
services:
  service1:
      images: « image »
      port:
        - « 80:90 »
      networks:
        - network1
      volumes:
        - dossierSrc:dossierDest
        - volumenommé:dossierDest
volumes:
   volumenommé:
```

Mount bind

Volume nommé docker